Human–Computer Interaction for Secure AI Tool Usage: Provenance-Aware, Risk-Adaptive Design Patterns Against Prompt Injection

[Author names withheld for review]
[Affiliation]
[corresponding e-mail]

Manuscript prepared 27 October 2026

Abstract

Tool-using AI agents can be hijacked by *indirect prompt injection*: instructions hidden in e-mails, documents or web pages that the agent reads. System-level defences that block every untrusted data flow lose utility on the many legitimate tasks that *must* use such data. Deployed agents therefore fall back on the human, usually through generic "approve this action?" dialogs, which usable-security research shows fail when they are frequent and uninformative.

We present Clear (Contextual, Lineage-aware, Escalating Agent-action Review), a human-centred mediation framework that decides *when* to involve the user and *what* to show them. It consolidates three design iterations into eight testable design patterns (P0–P7). Clear attributes every argument of every proposed tool call to its source; attribution is decoding-aware, so it resists provenance laundering. Contextual risk is scored from interpretable features and drives four escalation tiers. Dialogs pair source-to-action excerpts with a consequence preview computed in a sandbox, and the highest-risk tier adds a cognitive-forcing check. We prove an escalation floor that no attacker-controlled text can lower.

We replay 24,648 real trajectories of 22 LLM agents and five published defences in AgentDojo, recomputing the benchmark's own predicates (replay agreement 99.1%/99.4%). On held-out episodes Clear reduces attack success from 16.5% to 0.69% with a perfectly discerning user, and to 0.68% when every escalation is denied. It asks 51% fewer questions than confirm-every-action (0.38 vs. 0.79 per benign task; $p = 1.6 \times 10^{-116}$). We also simulate an attacker who encodes the target value, measuring residual success among attacks that beat the undefended agent. Lexical lineage then lets 5.9–7.9% succeed instead of 0.64%, and decoding-aware lineage restores 0.64%. Against detector-evading phrasing, Clear holds at 0.89%, while a provenance-free mediator stays at or above 9.3%. On 250 template-generated stress scenarios spanning code, cloud and finance, Clear matches the recall of two alternative HCI engines with far fewer false escalations of benign high-privilege actions (20% vs. 60% and 100%). On 8 argument-free fixtures it misses 2 of 4 attacks. A goal-drift signal borrowed from an earlier iteration was rejected on development data.

A parametric, habituation-aware user model (*not human data*) predicts lower end-to-end compromise than confirm-all in 100% of 243 settings and across heterogeneous user populations. We release all code, data, an instrumented console and a pre-registered 2×2 factorial user-study protocol, which has not yet been run.

Keywords: prompt injection; LLM agents; human-in-the-loop; usable security; provenance; security warnings; habituation; human–AI interaction

1 Introduction

Large language model (LLM) agents read external content and act on a user's behalf: they send e-mail, move money, share files and edit calendars. This combination creates a new class of

vulnerability, *indirect prompt injection* [25, 38, 47]. Text placed by a third party into data the agent processes—an e-mail body, a document, a web page, even a chat-channel name—is read as an instruction and redirects the agent. Benchmarks such as InjecAgent [64], AgentDojo [18] and ASB [66] show that frontier agents remain vulnerable. Adaptive and optimisation-based attacks break most model-level defences and detectors [15, 39, 45, 65].

The strongest defences move security out of the model and into a *system layer* that mediates tool calls, using capabilities and information-flow control [16, 19, 60], privilege policies [50, 67, 68] or counterfactual re-execution [28, 69]. These defences face an uncomfortable trade-off. Many legitimate tasks *require* the agent to act on untrusted data, such as paying the account number printed on a bill or replying to the sender of an e-mail. Strict policies therefore block legitimate work, and recent analyses argue that context-dependent flows cannot, in general, be classified without context [1, 59]. In practice, agents resolve the residual ambiguity by asking the user. Commercial agents and research systems alike rely on approval prompts [23, 41, 42].

Usable-security research has long warned that people are an unreliable last line of defence when warnings are frequent, generic and look alike [2, 17, 20, 21, 54]. Habituation sets in after only a few exposures and generalises across similar-looking notifications [6, 55, 56]. The first studies of AI-agent permissions point the same way. Participants approved many actions outside their request when asked per action [62]. Practitioners report that what lies behind an action is invisible to them [48]. Permission systems report fatigue as a central design constraint [9, 68]. *What* the human sees when an agent escalates, and *how often* the human is asked, are therefore security-critical design decisions. They have received little systematic attention.

This paper. We design and evaluate CLEAR, a human-centred mediation layer that answers both questions.

When to ask. CLEAR keeps a provenance ledger of everything the agent has read and attributes every argument of a proposed tool call to the user's request, a trusted record, a specific piece of external content, or nothing. It combines this *argument-level lineage* with the tool's harm potential, the egress of private data, deviation from a user-editable *intent envelope*, and the instruction-likeness of untrusted content. The result is an interpretable risk score that drives four escalation tiers: allow, notice, confirm, and force.

What to show. Confirmation dialogs show where each value came from, quoting the source passage (a *source-to-action* excerpt). They also show what the action would change, computed by executing it on a copy of the environment, and why it is suspect. The highest-risk tier uses a deliberately different interaction: a cognitive-forcing check [11] asking where a value came from. This keeps habituation to routine dialogs from transferring to rare, dangerous ones.

Three iterations, one framework. CLEAR consolidates three design iterations of this project. An early prototype contributed the source-to-action framing and a factorial study design. A second iteration proposed decoded-payload provenance, goal-drift scoring and counterfactual previews, but validated them only with a simulator whose effect sizes were hard-coded. We re-implemented every borrowed idea and evaluated it on real trajectories. We adopted decoding-aware lineage and a *computed* consequence preview, rejected goal drift, and audited the earlier evidence (Section 8).

Contributions.

1. A structured review of 2023–2026 prompt-injection defences, organised by the *role of the human* and the *evidence* the human receives. It identifies the gap at the intersection of system-level mediation and the human factors of escalation (Section 2).
2. The CLEAR framework: eight testable design patterns (P0–P7), a formal risk and escalation model, an *escalation floor* that attacker-controlled text cannot lower (Proposition 1), and its extension to laundered values (Proposition 2) (Section 4).

3. An open implementation. It comprises the mediator, an AgentDojo-compatible replayer, baselines (including two alternative HCI defence engines), an instrumented console that doubles as a study apparatus, and 62 automated tests.
4. The, to our knowledge, largest environment-grounded evaluation of a human-in-the-loop prompt-injection defence. It covers 24,648 real trajectories from 22 LLMs and five published defences, and reports security, utility *and user burden*, with significance tests, ablations, a detector-evading attacker, a provenance-laundering attacker, a cross-benchmark stress test and latency (Sections 5 and 6).
5. A habituation-aware signal-detection model, robust to heterogeneous user populations, that turns escalation decisions into end-to-end risk. We use it to derive the hypotheses of a pre-registered 2×2 factorial user study (selectivity × evidence), whose protocol and apparatus we release (Sections 6.8 and 7).
6. A reproducible audit of simulated "user-study" evidence produced in an earlier iteration. It shows how hard-coded effect sizes can masquerade as findings, and derives reporting rules for simulated HCI-security evidence (Section 8).

Scope and honesty statement. All agent behaviour analysed here is *real*: it consists of recorded trajectories released with AgentDojo, replayed in AgentDojo's environments. No data were collected from human participants. Every result about people comes from an explicitly parametric simulation and is labelled as such. The user study is a protocol and has not been conducted.

2 Background and related work

2.1 Prompt injection against tool-using agents

Prompt injection was first demonstrated as goal hijacking of instruction-following models [47]. It became an application-security problem once retrieved content could carry it [25, 37]. Liu et al. [38] formalised attacks and defences. Benchmarks moved the threat to agents: InjecAgent [64], BIPIA [63], ASB [66] and AgentDojo [18]. AgentDojo is a dynamic environment with banking, Slack, travel and workspace suites; 97 user tasks; 629 security cases per attack; and stateful utility and security predicates. It also publishes full traces of many LLM agents under several attacks and defences, which we reuse. Follow-up work shows that argument-level scoring matters [49], that personal data can be exfiltrated through simple injections [3], and that automated and reinforcement-learning attacks transfer across tasks [15, 30].

2.2 Model- and prompt-level defences

Prompt transformations mark the provenance of data inside the prompt, as in spotlighting [29]. Training-based defences teach models to ignore instructions in data: the instruction hierarchy [58], StruQ [12], SecAlign [14] and Meta SecAlign [13]. Detectors and sanitisers filter tool outputs [51]. These approaches reduce attack success but give no guarantee. White-box and adaptive attacks break them [15, 45, 65], and static benchmarks overestimate them [39].

2.3 System-level defences

CaMeL [19] extracts control and data flow from the trusted query and enforces capability policies on tool calls. FIDES [16] and f-secure [60] apply information-flow control. LLMbda gives such provenance-based defences a formal semantics [22]. Progent [50], ClawGuard [67], ToolFence [36], MiniScope [68], CSAgent [24] and PFI [33] enforce least privilege derived from the user's task. MELON [69], AttriGuard [28], Task Shield [31], IPIGuard [5] and DRIFT [35] test whether an

action is caused by the user's intent. Beurer-Kellner et al. [8] catalogue architectural patterns with provable resistance. These systems report impressive results on AgentDojo, but under static attacks [43]. Their utility costs on context-dependent tasks are well documented: CaMeL solves 77% of AgentDojo tasks with guarantees against 84% undefended [19], and context-suppressing defences fail when context is essential [59]. Abdelnabi et al. [1] argue, through contextual integrity [44], that this tension is fundamental. Whenever such systems defer to people ("expansion requires explicit approval" [50]; "user-confirmed rule set" [67]), the human interaction is left unspecified and unmeasured.

2.4 Human oversight and permissions for AI agents

Magentic-UI asks users to approve actions labelled irreversible, using an LLM judge for ambiguous cases [42]. Plan-then-execute interaction affects trust and team performance [27]. A 2026 survey of 21 agent-permission systems and five commercial agents identifies gaps in user-level permission handling [41]. A walkthrough of 18 commercial agents finds bundled permissions and default opt-ins [23]. Wu et al. [61] predict users' permission decisions from context. Janus explores user-involved permission assistants with synthetic responders [9]. Yan [62] compared per-action approval, automated review and user-authored rules with 113 participants and found that many overreach actions were *approved by users*. Salerno et al. [48] report that practitioners struggle to see what lies behind an agent's actions. Authenticated delegation frameworks address *who* an agent acts for [52].

The closest work to ours is Su et al. [53]. They propose Risk Cards, which show source-trust labels, data-sensitivity chips and a risk badge, and evaluate them on AgentDojo traces with a deterministic information-exposure proxy. CLEAR differs in four ways. It contributes a formal, ablated risk model with an attacker-independent escalation floor. It recomputes *environment outcomes* (utility and security) rather than an exposure proxy. It treats *interruption burden* as a first-class metric and compares against system-level and published defences with paired tests. And it models habituation explicitly, adding a differentiated forcing tier.

2.5 Warnings, habituation and reliance

Warning effectiveness depends on design: active warnings outperform passive ones [20], and field click-through rates vary widely with the user experience of the warning [2]. Frequent false alarms train users to dismiss warnings [54], and install-time permission dialogs are rarely read [21]. Neuro-physiological studies show that habituation sets in after a few exposures and can be reduced by polymorphic designs [6, 55]. It also generalises from frequent notifications to rare warnings that look similar [56]. In human–AI decision-making, explanations can increase reliance on wrong advice [7]. They reduce overreliance when they make verification cheap [57]. Cognitive forcing reduces overreliance at a cost in perceived usability [11]. These findings frame automation trust as a matter of appropriate reliance [4, 34, 46]. They motivate three CLEAR principles: ask rarely, show *verifiable* provenance rather than an AI's opinion, and reserve forcing for the rare highest-risk case.

2.6 Gap

Table 1 positions prior work by the human's role. No prior work jointly decides *when* to involve the human using provenance-aware contextual risk with a formal floor, designs *what* the human sees at argument level, *and* evaluates the resulting human–machine system on real attacked trajectories in terms of security, utility and burden.

Table 1: Where is the human? Representative prompt-injection defences by human role and evidence. "Burden" marks whether the human cost of the defence is reported.

Human role	Representative work	What the human sees	Burden
Absent (model)	spotlighting, instruction hierarchy, StruQ, SecAlign, Meta SecAlign, detectors	nothing	n/a
Absent (system)	CaMeL, FIDES, f-secure, MELON, Task Shield, IPIGuard, DRIFT, AttriGuard	nothing (calls fail)	no
Pre-authorisation	Progent, ClawGuard, MiniScope, ToolFence, user-authored policies	rules / scopes	partly
Runtime confirmation	Magentic-UI, Janus, Risk Cards, commercial agents	call or card with labels	proxy
Clear (ours)	envelope + risk-adaptive escalation + audit	argument lineage, reasons, forcing	**yes**

3 Threat model and design goals

Setting. A user issues a natural-language request to an LLM agent, which iteratively proposes tool calls. A *mediator* sits at the tool boundary. It sees the user's request, every tool output with a manifest-declared trust label, and each proposed call before execution. It does *not* trust or inspect the model's reasoning.

Adversary. The attacker controls the content of some untrusted sources: e-mails they send, documents they share, web pages, reviews, chat messages and channel names. The attacker can also adapt that text to the mediator, including evading any lexical detector. The attacker cannot modify the user's request, the tool manifest, trusted system records (for example, the user's own account data), or the mediator. The LLM is treated as fully compromisable: any call may be attacker-induced.

Goals. (G1) Prevent attacker-induced side effects. (G2) Preserve the utility of legitimate tasks, including tasks that legitimately consume untrusted data. (G3) Minimise and budget human interruptions. (G4) When the human is asked, provide evidence that makes the correct decision easy to verify. (G5) Remain robust when text-based signals are evaded. (G6) Add negligible latency.

Out of scope. Attacks that manipulate only the agent's *textual answer* (for example, a recommendation) without a tool call cannot be stopped at the tool boundary. We measure and report them separately. Compromise of the tool implementations themselves and multi-agent delegation are also out of scope.

4 The Clear framework

4.1 Design patterns

Clear is organised as eight testable design patterns (Figure 1).

P0 Declared capabilities. Each tool is annotated once by its developer, analogous to mobile-OS permission levels. The annotation records a capability class, a sensitivity $s(f) \in [0, 1]$, reversibility, target/content/value arguments, whether outputs are *trusted* (system of record) or

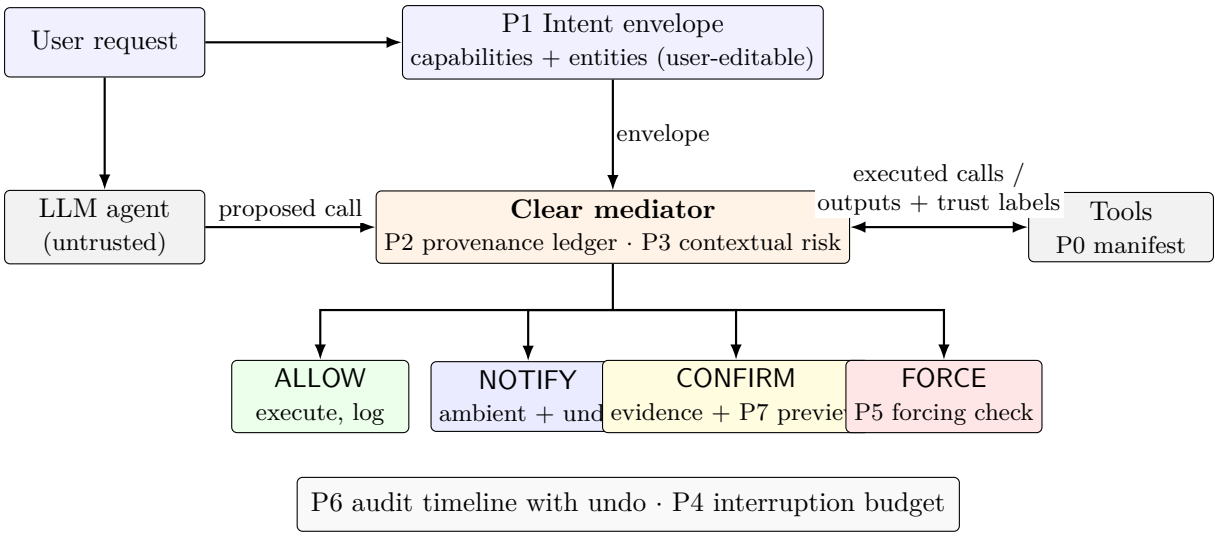


Figure 1: Clear architecture. The mediator never trusts the model. It reasons over the user's request, manifest-labelled tool outputs and the proposed call, and routes each call to one of four escalation tiers.

untrusted (may contain third-party text), and whether outputs are private. Unknown tools fail closed ($s = 0.8$, irreversible).

P1 Intent envelope. At task start, the request is compiled into the capability classes and entities it plausibly needs. The envelope is shown to the user, who can widen or narrow it before the agent runs. Our default compiler is deterministic and LLM-free (a verb lexicon and entity patterns), so it cannot itself be injected. Explicit delegation ("do the actions in David's e-mail") is detected and weakens, but does not remove, the deviation signal.

P2 Argument-level lineage. Every argument value is attributed to the user's request, a trusted record, specific external sources, or nothing. Trust requires positive evidence: values that appear nowhere are treated as weakly tainted (*fail-closed endorsement*). Attribution is *decoding-aware* (v2). Untrusted text is canonicalised (NFKC, invisible characters removed, homoglyphs folded, "[at]/[dot]" resolved), and payloads hidden by base64, hex, percent- or HTML-encoding or ROT13 are decoded and indexed. A value laundered through such an encoding is therefore still traced to its source, and hiding text from the human reader is itself counted as an injection cue.

P3 Contextual risk. Interpretable features are combined into an expected-harm proxy (Section 4.2). Each nonzero feature yields a plain-language reason that is shown to the user.

P4 Risk-adaptive escalation. Four tiers: ALLOW (silent, logged), NOTIFY (ambient notice with undo), CONFIRM (blocking dialog with provenance) and FORCE (blocking dialog with cognitive forcing). Irreversible actions are never executed silently above ALLOW. The thresholds set an *interruption budget*.

P5 Differentiated forcing. FORCE dialogs look and behave differently from routine ones. The approve button stays disabled until the user answers where the critical value came from, and a wrong answer is corrected. Differentiating the *mode of interaction* is how warning research mitigates the generalisation of habituation [6, 56]. Forcing improves decisions at a usability cost [11], so Clear reserves it for the highest tier.

P6 Audit and undo. An ambient timeline lists every executed action and its tier, and reversible actions can be undone.

P7 Computed consequence preview. Every escalated action is executed on a *copy* of the environment in its current state. The resulting state difference is shown in plain language ("Adds a bank transaction: recipient=US13…, amount=0.01"; "Removes a file: sales-report-Q1-2024.xlsx"), together with whether the action can be undone. This makes counterfactual previews concrete: they are computed, not asserted, and they expose side effects the user may not anticipate. For example, creating a calendar event in AgentDojo also e-mails every participant.

4.2 Formal model

Let $\mathcal{C}_t = (c_1, \ldots, c_n)$ be the sources recorded up to step t. Each source has a trust label $\ell(c) \in \{\mathsf{U}, \mathsf{T}, \mathsf{E}\}$ (user, trusted record, external) and a privacy flag. For a value v (an argument or an element of a list argument), let $\mathrm{norm}(\cdot)$ be a normalisation (case, whitespace, separator and URL-scheme insensitive). Define the attribution set $A(v) = \{c \in \mathcal{C}_t : \mathrm{norm}(v) \sqsubseteq \mathrm{norm}(c)\}$, where $\sqsubseteq$ denotes substring containment, and the *origin*

$$o(v) = \begin{cases} \mathsf{U} & \exists c \in A(v) : \ell(c) = \mathsf{U} \\ \mathsf{T} & \text{else if } \exists c \in A(v) : \ell(c) = \mathsf{T} \\ \mathsf{E} & \text{else if } A(v) \neq \emptyset \\ \bot & \text{otherwise (unattributed).} \end{cases} \tag{1}$$

For a proposed call $a = (f, \mathbf{x})$ with target arguments T_f, value arguments V_f and content arguments K_f, CLEAR computes five features.

Taint $\tau(a) = \max\big(\max_{v \in T_f} \rho_T(o(v)),\ \max_{v \in V_f} \rho_V(o(v)),\ \max_{z \in Z(K_f)} \rho_K(o(z))\big)$. Here $\rho_T(\mathsf{E}) = 1$, $\rho_T(\bot) = 0.5$, $\rho_V(\mathsf{E}) = \rho_K(\mathsf{E}) = 0.4$, and all other levels are zero. $Z(\cdot)$ extracts distinctive tokens (URLs, addresses, account numbers, codes) from outgoing content.

Exfiltration $\kappa(a) = \mathbb{1}[\mathrm{egress}(f) \wedge \neg\mathrm{endorsed}(T_f) \wedge \text{payload carries private-source data}]$.

Deviation $\delta(a) = \mathbb{1}[\mathrm{cap}(f) \notin \mathrm{Env}]$, or 0.5 under explicit delegation.

Instruction-likeness $\iota(a) = \max(\iota_{\mathrm{attr}},\ \omega\, \iota_{\mathrm{ctx}})$. ι_{attr} is the maximum instruction-likeness of the sources the arguments are attributed to. ι_{ctx} is the maximum over all untrusted sources read in the task. Instruction-likeness is computed by a transparent lexicon of attack strategies (context ignoring, fake authority, sequencing, fake completion, addressing the model).

These features are combined by a noisy-OR into the probability that the action is unintended, and weighted by harm potential:

$$P(a) = 1 - \prod_{i \in \{\tau, \kappa, \delta, \iota\}} (1 - w_i x_i(a)), \qquad R(a) = s(f)\, P(a). \tag{2}$$

The tier is FORCE if $R \geq \theta_F$, CONFIRM if $R \geq \theta_C$, and NOTIFY if $R \geq \theta_N$ (promoted to CONFIRM if f is irreversible); otherwise it is ALLOW. All weights and thresholds were fixed a priori: $w_\tau = 0.9$, $w_\kappa = 0.8$, $w_\delta = 0.7$, $w_\iota = 0.6$, $\omega = 0.5$, $\theta_N = 0.15$, $\theta_C = 0.35$, $\theta_F = 0.7$. They were not fitted to outcomes (Section 5.6).

Proposition 1 (Escalation floor)**.** *Let $a = (f, \mathbf{x})$ be a call with some target argument whose origin is external, $o(v) = \mathsf{E}$, so that $\tau(a) = 1$. (i) If $s(f) \geq \theta_C / w_\tau$, then a is escalated to at least CONFIRM, and if $s(f) \geq \theta_F / w_\tau$, to FORCE. (ii) If f is irreversible and some distinctive token in its content is external ($\rho_K = 0.4$), then a reaches at least CONFIRM whenever $s(f) \geq \theta_N / (0.4\, w_\tau)$. Both bounds hold for every value of ι, κ and δ, and therefore for every attacker-controlled text.*

Proof. Each factor $(1 - w_i x_i)$ in Equation (2) lies in $[0, 1]$, so $P(a) \geq 1 - (1 - w_\tau \tau(a)) = w_\tau \tau(a)$ and $R(a) \geq s(f)\, w_\tau\, \tau(a)$. For (i), $\tau = 1$ gives $R \geq s\, w_\tau \geq \theta_C$ (respectively θ_F). For (ii), $\tau \geq 0.4$ gives $R \geq 0.4\, s\, w_\tau \geq \theta_N$, and irreversibility promotes NOTIFY to CONFIRM. The attacker controls only untrusted text. This can change ι, and can make additional values external, but by definition it cannot make an external value user-endorsed or trusted. Since R is non-decreasing in every feature, attacker text cannot lower the bound. □

With the default parameters, (i) covers every manifest tool with $s \geq 0.39$ (CONFIRM) and $s \geq 0.78$ (FORCE). This includes all payment, credential, e-mail, messaging, sharing and publishing tools. (ii) covers irreversible tools with $s \geq 0.42$. The floor is *text-independent* but not *attack-complete*. Attacks whose targets are user-endorsed or come from trusted records ("delete the largest file") rely on δ and ι, and output-only manipulations are out of reach of any tool-boundary mediator. We quantify both in Section 6.

Provenance laundering. An adaptive attacker can try to break the premise of Proposition 1: plant the target value in disguised form and rely on the LLM to decode it, so that the value appears in *no* source. Let $\mathcal{D}$ be the set of transformations the decoder inverts (canonicalisation and the decodings listed in P2). Decoding-aware attribution evaluates $A(v)$ against every source c *and* its decoded variants $\{d(c)\}_{d \in \mathcal{D}}$.

Proposition 2 (Laundering closure)**.** *If an attacker-chosen value v reaches the agent only through untrusted content in which it appears as $t(v)$ for some t with $t^{-1} \in \mathcal{D}$, then decoding-aware attribution yields $o(v) = \mathsf{E}$. Proposition 1 therefore applies to v unchanged. Under lexical attribution, $o(v) = \bot$ and the floor weakens to $R \geq s\, w_\tau\, \rho_T(\bot) = 0.45\, s$.*

Proof. $t^{-1}(t(v)) = v$ is among the indexed variants of the carrying source c, so $c \in A(v)$ and $\ell(c) = \mathsf{E}$. By the attacker model, v is absent from user and trusted sources, so the origin is E. The weakened bound follows from Equation (2) with $\tau = \rho_T(\bot) = 0.5$. □

The weakened bound still escalates payments and credentials ($s = 1$: $R \geq 0.45 \geq \theta_C$). Messaging and sharing tools ($s = 0.8$: $R \geq 0.36$) and lower-sensitivity tools, however, fall below the forcing tier or reach only a notice. Experiment E1 (Section 6.6) measures this gap on real trajectories. Encodings outside $\mathcal{D}$, such as a value the attacker asks the model to compute, remain a limitation.

4.3 Interface

Figure 2 shows the console. Before the agent starts, the intent envelope appears as toggleable capability chips with the entities the request names. Agent activity is a timeline in which untrusted outputs are tagged *external content*, and text addressed to the agent is highlighted with a banner. A CONFIRM dialog summarises the action in plain language and shows a risk meter and each argument with its provenance chip (*from your request*, *from a trusted record*, *from external content* with the source, or *not found in any source*). For external values, the dialog also shows the source passage with the value highlighted. It then shows the computed consequence preview (P7) and the reasons. A FORCE dialog adds the forcing question. All trace content is rendered as text only, never as HTML. Envelope edits re-score calls client-side with the same model.

4.4 Implementation

CLEAR is a dependency-light Python package. A `Mediator` exposes `start_task`, `observe_result`, `assess` and `decide`, so it can wrap any agent loop. The AgentDojo adapter streams recorded or live messages through it. The console is a standard-library HTTP server with a vanilla-JavaScript client and supports the four conditions of a 2×2 factorial study plus a no-mediation arm (Section 7). Configuration, the tool manifest and every design decision made on development data are versioned. The test suite (62 tests) includes a randomised property test of Proposition 1, laundering tests for Proposition 2, a label-blindness test for the baseline engines, end-to-end mediation, UI logging and input validation, a check that participant payloads do not leak ground-truth labels, and integration tests against real traces.

5 Evaluation methodology

5.1 Data: real agent trajectories

AgentDojo's public repository (pinned commit `089ed46`) contains the complete message traces of 22 LLM agents under its attacks. These span GPT-3.5/4/4o, Claude 3–3.7, Gemini 1.5/2.0, Llama 3/3.3, Command-R and Meta SecAlign. It also contains traces of GPT-4o, Llama-3.3-70B, Claude-3-Sonnet and Meta-SecAlign-70B protected by published defences: sandwich prompting, spotlighting, tool filtering and a DeBERTa injection detector. We use every benign run and every run under the `important_instructions` attack. For GPT-4o we additionally use `tool_knowledge`, `ignore_previous`, `direct` and `injecagent`. In total: 29 pipelines, 24,648 episodes (21,893 attacked, 2,755 benign) and 90,248 tool calls, of which 23,793 are side-effecting.

5.2 Trace replay

For every episode we stream the trace through each mediator *online*: each call is assessed before its result is observed. Each mediation policy yields a set of calls that would not have executed. We re-execute the remaining calls in AgentDojo's own environment (with the episode's injections) and evaluate the benchmark's own utility and security predicates. The benchmark version is selected per pipeline by agreement on a probe sample. Undefended replays agree with AgentDojo's stored labels on 99.1% (utility) and 99.4% (security) of episodes; the lowest pipeline is Claude-3-Opus at 84.3% utility, and Section 10 reports a sensitivity analysis excluding low-fidelity pipelines. For GPT-4o, our replayed attack success rates for the undefended agent and the four published defences agree with the published AgentDojo figures to within 1 percentage point (Table 3).

Utility is scored *conservatively*. If any non-injected call is stopped, the task counts as failed, even if the benchmark predicate would still pass. Replay cannot observe how an agent would have adapted after a denied call; we discuss this in Section 10.

5.3 Ground-truth labels for calls

A side-effecting call in an attacked episode is labelled *injected* if either of two independent criteria holds. (a) *Signature*: its function belongs to the injection task's ground-truth calls and either carries an attacker-specific value (one absent from the user prompt and user-task ground truth) or the function is absent from the user task. (b) *Causal necessity*: removing this call alone makes AgentDojo's security predicate fail. On side-effecting calls of successful attacks the two criteria agree on 95.5% of calls (Cohen's $\kappa = 0.91$). In total 5,234 calls are labelled injected.

5.4 Policies and human-response modes

All tool-call mediators run on the same substrate.

- **No defence.**
- **Injection detector (abort).** Aborts the episode at the first untrusted output whose instruction-likeness is ≥ 0.5. This mirrors AgentDojo's DeBERTa detector, which aborts the agent.
- **Task-scoped allowlist.** Denies calls whose capability is outside the envelope, in the spirit of Progent and ClawGuard.
- **Strict information-flow policy.** Denies side-effecting calls with any non-endorsed target, value or planted token, or with private-data egress. This follows CaMeL/FIDES-style policies.
- **Confirm-all.** Asks about every side-effecting call; this is the de-facto industry pattern.
- **Clear.** Asks about CONFIRM/FORCE calls only.

For the two human-in-the-loop policies we report two bounding response modes: *ideal user* (denies exactly the injected calls) and *deny escalations* (no human: every escalation is denied, fail-closed). A rubber-stamping user would reproduce the undefended result. Section 6.8 studies the space between these bounds.

5.5 Metrics and statistics

ASR is the fraction of attacked episodes whose security predicate holds after mediation. **UA** and **BU** are utility under attack and benign utility. **Dialogs per benign task** measures user burden. At the call level we report **escalation recall** (injected side-effecting calls reaching CONFIRM/FORCE), the **false escalation rate (FER)** on benign side-effecting calls, AUROC of R, and per-call latency. Confidence intervals are 95% cluster-bootstrap percentiles (2,000 resamples; clusters are model × suite × user task). Paired binary outcomes use exact McNemar tests with Holm correction within each family. Paired counts use the Wilcoxon signed-rank test with matched-pairs rank-biserial correlation.

5.6 Development split and frozen configuration

The only data inspected during design were GPT-4o traces on the banking and Slack suites. Five changes were made on this split and are logged in the repository. Two corrected manifest annotations: Slack channel and user names are third-party-authored. One added URL canonicalisation to attribution. One added the planted-token content feature. One fixed a labelling bug. No weight or threshold was changed. The configuration was committed before any held-out result was computed. All main results use *held-out* episodes: travel and workspace for GPT-4o, and all suites for the other 21 models (14,865 attacked, 2,096 benign).

5.7 Combined version and additional experiments

Second freeze point. Candidate features from the earlier iterations were evaluated on the same development split with a decision rule fixed in advance: adopt a feature if it does not raise ideal-user ASR and does not raise dialogs per benign task by more than 10%.

- *Decoding-aware lineage* left every development metric unchanged and was adopted (CLEAR v2, committed before any held-out re-evaluation).
- A lexical *goal-drift* feature γ (cosine mismatch between the request and the action; weight 0.3) raised dialogs per benign task by 22% without lowering ASR, and was rejected.

The full replay was re-run with v2 and its variants. All v1 results reproduced exactly.

E1: provenance laundering. We take every held-out episode in which the recorded attack succeeded (2,444 episodes). Inside the tool outputs we re-encode the attacker-specific argument values in five ways (base64, hex, Cyrillic homoglyphs, zero-width splitting, "[at]/[dot]") or base64-encode the whole injection block. The agent's recorded calls are unchanged: they carry the decoded value, which is the attacker's goal. This is a worst case for the defender, since encoding usually lowers an injection's success. Executed calls and environments are unchanged, so ideal-user outcomes are exact; we look them up among the stored outcomes for every subset of denied injected calls. Two robustness gaps of the v2 decoder surfaced in this held-out experiment. Payloads glued to serialised "\n" escapes were not detected, and decoded payload text was not scored for instruction-likeness. We fixed both in a *post-hoc* v2.1, which we report separately from the frozen v2.

Table 2: Held-out evaluation (22 LLM agents; development split excluded; `important_instructions` attack). Point estimates with 95% cluster-bootstrap confidence intervals (clusters: model × user task).

Policy	ASR %	Utility under attack %	Benign utility %	Dialogs / benign task
No defence	16.5 [15.4, 17.5]	44.4 [42.3, 46.3]	54.4 [52.2, 56.4]	0.00 [0.00, 0.00]
Injection detector (abort)	0.1 [0.0, 0.2]	5.2 [4.3, 6.1]	54.4 [52.2, 56.4]	0.00 [0.00, 0.00]
Task-scoped allowlist (auto)	10.7 [9.8, 11.6]	44.5 [42.5, 46.6]	51.2 [49.0, 53.3]	0.00 [0.00, 0.00]
Strict info-flow policy (auto)	2.6 [2.3, 2.9]	32.4 [30.5, 34.3]	31.8 [29.8, 33.8]	0.00 [0.00, 0.00]
Confirm-all (ideal user)	0.5 [0.4, 0.7]	48.8 [46.7, 50.9]	54.4 [52.2, 56.4]	0.79 [0.74, 0.85]
Confirm-all (deny all)	0.5 [0.4, 0.6]	22.8 [20.9, 24.6]	22.1 [20.3, 23.9]	0.79 [0.74, 0.85]
CLEAR (ideal user)	0.7 [0.6, 0.8]	48.8 [46.7, 50.8]	54.4 [52.2, 56.4]	0.38 [0.35, 0.42]
CLEAR (deny escalations)	0.7 [0.5, 0.8]	32.5 [30.5, 34.5]	35.2 [33.1, 37.2]	0.38 [0.35, 0.42]
CLEAR v2 (ideal user)	0.7 [0.6, 0.8]	48.8 [46.7, 50.8]	54.4 [52.2, 56.4]	0.38 [0.35, 0.42]
CLEAR v2 (deny escalations)	0.7 [0.5, 0.8]	32.5 [30.5, 34.5]	35.2 [33.1, 37.2]	0.38 [0.35, 0.42]

E2: cross-benchmark stress test. To probe domains and attack families absent from AgentDojo, we use the call-level scenarios of the two earlier iterations. *PROV-Guard-250* comprises 250 template-generated single-call scenarios in five domains (e-mail, cloud drive, banking, code/IDE, calendar) and five attack families (direct override, context framing, obfuscated payload, cross-tool chain, goal mimicry). *Trust-Boundary-8* comprises 8 hand-authored fixtures without tool arguments. Tools are annotated in a separate stress manifest. We compare CLEAR v1/v2 with the two earlier engines, run unmodified and *blinded* to ground-truth fields (a test checks label-blindness), and with confirm-all and the lexical detector. Two properties of PROV-Guard-250 limit what it can show. First, a single provenance rule (a critical value appears in external content and not in the request) separates all 150 attacks from all 100 benign cases. Second, every attack uses an external or high-privilege tool. We therefore report recall per family and the false escalation rate on the 25 benign high-privilege calls, and never treat the benchmark as agent-level ASR.

E3: heterogeneous users. The simulated-user analysis (Section 6.8) is repeated with users drawn from a mixture of three expertise strata (novice, intermediate, security-aware). Each stratum is a distribution over sensitivity, criterion and habituation parameters with 15% individual jitter, and three mixtures are compared.

6 Results

6.1 RQ1: Security

Table 2 and Figure 3 summarise the held-out results. Without a defence, 16.5% of attacked episodes succeed. CLEAR reduces this to 0.69% [0.56, 0.83] with an ideal user and to 0.68% when every escalation is denied. The reduction is significant against no defence (2,344 vs. 0 discordant pairs), against the task-scoped allowlist (10.7%; 1,481 vs. 0) and against the strict information-flow policy (2.60%; 285 vs. 2; all Holm-adjusted $p < 10^{-80}$).

Two baselines achieve lower ASR on this attack. Confirm-all with an ideal user reaches 0.54%: it asks about every side-effecting call, so it also catches the 23 episodes CLEAR lets through ($p = 2.4 \times 10^{-7}$). The lexical injection detector reaches 0.09%, because its patterns match the `important_instructions` template. It does so by aborting: utility under attack collapses to 5.2%, and Section 6.4 shows the protection vanishes once the template changes.

Of CLEAR's 103 residual successes, 78 come from a single *output-only* injection task. That task makes the travel agent recommend a hotel without any tool call, so no tool-boundary mediator can stop it; it also accounts for most of confirm-all's residual. Excluding it, CLEAR's ideal-user ASR is 0.17%.

Table 3: Published defences replayed alone and with CLEAR (all four suites; `important_instructions`). "Dialogs" are per benign task. The ASRs of the defences alone reproduce the published AgentDojo figures (GPT-4o: 47.7, 27.8, 41.7, 6.8, 8.0%).

		Published defence alone		+ CLEAR, ideal user		+ CLEAR, deny escalations	
Model	Defence	ASR %	BU %	ASR %	Dialogs	ASR %	BU %
GPT-4o	none (undefended)	48.6	70.1	0.6	0.56	0.6	39.2
GPT-4o	sandwich (repeat user prompt)	28.1	84.5	0.8	0.69	0.8	52.6
GPT-4o	spotlighting (delimiting)	41.8	72.2	0.0	0.53	0.0	41.2
GPT-4o	tool filter	6.8	72.2	1.3	0.43	1.3	44.3
GPT-4o	DeBERTa PI detector	7.9	41.2	0.0	0.20	0.0	27.8
Llama-3.3-70B	none (undefended)	23.1	57.7	0.5	0.43	0.5	37.1
Llama-3.3-70B	sandwich (repeat user prompt)	9.1	58.8	1.1	0.66	1.0	38.1
Meta-SecAlign-70B	none (undefended)	2.2	77.3	0.0	0.59	0.0	49.5
Meta-SecAlign-70B	sandwich (repeat user prompt)	2.1	85.6	0.0	0.55	0.0	51.5

6.2 RQ2: Burden and utility

CLEAR asks 0.38 questions per benign task against 0.79 for confirm-all, a 51% reduction. Every one of the 612 benign tasks with a difference favours CLEAR (Wilcoxon $p = 1.6 \times 10^{-116}$, $r_{rb} = -1$). 74% of benign tasks complete without any dialog, against 54% for confirm-all. In attacked episodes the numbers are 0.66 versus 0.93. With an ideal user, neither policy loses benign utility (54.4%, equal to no defence). Utility under attack *rises* from 44.4% to 48.8%, because removing injected actions often restores the user's task.

Without a human, fail-closed CLEAR keeps 35.2% benign utility. This is higher than confirm-all-deny (22.1%; 274 vs. 0 discordant tasks) and the strict information-flow policy (31.8%; 85 vs. 15; both $p < 10^{-12}$). It remains well below the undefended 54.4%: the legitimate actions that CLEAR escalates are exactly those that need a human.

6.3 Composition with published defences and models

Table 3 replays CLEAR on top of the published defences. Model-/prompt-level defences reduce ASR but leave between 2.1% (Meta SecAlign) and 41.8% (spotlighting) of attacks successful. Adding CLEAR brings every pipeline to $\leq$1.3% with an ideal user. The detector pipeline shows the cost of aborting: its benign utility is 41.2%. Across all 22 models (Figure 8, appendix), CLEAR's ideal-user ASR never exceeds 3.0% (Claude-3.5-Sonnet-0620), against an undefended range of 0.9–56.8%.

6.4 Attack types and a detector-evading attacker

Table 4 varies the attack (GPT-4o, all suites). The lexical detector is near-perfect on the two templates its patterns describe. It offers no protection against `ignore_previous`, whose AgentDojo payload misspells "instructions", or against `direct` ("TODO:" followed by the goal). CLEAR brings every attack to $\leq$0.8%, and to $\leq$1.1% even without its own instruction signal ($-\iota$), as predicted by Proposition 1.

To model an *adaptive* attacker who phrases injections so that no lexical cue fires, we set every text-derived signal to zero. Ideal-user outcomes can be recomputed exactly for any tier assignment from the stored outcomes of every subset of denied injected calls. These offline recomputations reproduce the online tiers on 100% of calls. Under this attacker, the detector's ASR reverts to the undefended 16.5%. CLEAR's rises only from 0.69% to 0.89%. A provenance-free mediator, which keeps δ but drops τ, κ, cannot go below 9.3% at *any* threshold (Figure 4). This is the central finding of the ablation. On the known template, the provenance-free mediator is cheaper: 0.63% ASR at 0.11 dialogs per task. That efficiency comes from the lexical signal and disappears under evasion. Argument provenance is what buys robustness.

Table 4: ASR (%) by attack type for GPT-4o (629 security cases each). $-\iota$: CLEAR without its instruction-likeness feature.

Attack	No defence	Lexical detector	CLEAR $-\iota$ (ideal)	CLEAR (ideal)	CLEAR (deny)
important instructions	48.6	0.0	1.0	0.6	0.6
tool knowledge	34.8	0.2	1.1	0.8	0.8
ignore previous	5.4	5.4	0.0	0.0	0.0
direct (TODO)	3.7	3.7	0.0	0.0	0.0
InjecAgent	5.7	0.2	0.0	0.0	0.0

Table 5: Ablations on held-out episodes. Recall: share of injected side-effecting calls escalated to CONFIRM or FORCE; FER: false escalation rate on benign side-effecting calls.

Variant	ASR ideal %	ASR auto %	BU auto %	Dialogs/benign	Recall %	FER %	AUROC
CLEAR (full)	0.7	0.7	35.2	0.38	98.7	58.9	0.858
– provenance (τ, κ)	3.4	3.3	50.4	0.06	82.2	32.9	0.892
– taint (τ)	2.7	2.7	45.4	0.15	85.2	42.5	0.867
– exfiltration (κ)	0.7	0.7	35.4	0.38	98.7	57.9	0.864
– envelope deviation (δ)	1.4	1.4	35.4	0.37	95.7	58.1	0.833
– instruction signal (ι)	0.9	0.9	36.1	0.37	97.7	44.8	0.850
single threshold (no tiers)	0.6	0.6	32.0	0.46	99.2	71.9	0.858
oracle intent envelope	0.7	0.7	35.4	0.38	98.8	58.4	0.874
CLEAR v2 (+ decoding-aware lineage)	0.7	0.7	35.2	0.38	98.7	58.9	0.858
v2 + goal drift γ (rejected)	0.7	0.6	33.3	0.48	98.9	63.6	0.856
v2 – instruction signal (ι)	0.9	0.9	36.1	0.37	97.7	44.8	0.850

6.5 Ablations and call-level behaviour

Table 5 isolates each component. Removing provenance raises ASR fivefold (0.7 → 3.4%; 396 vs. 0 discordant episodes). Removing taint alone gives 2.7%, and removing envelope deviation doubles ASR (1.4%). Removing the instruction signal costs +0.2 points. Removing exfiltration has no effect on AgentDojo: its injection goals with private-data egress also have external targets, so τ already covers them. We keep κ for egress to user-named recipients, which AgentDojo does not exercise. A single threshold without tiers catches 12 more episodes but raises burden by 19% and lowers fail-closed utility. Replacing the lexical envelope compiler with an oracle built from ground-truth calls leaves ASR unchanged and reduces dialogs by only 2%, so the simple compiler is not a bottleneck.

CLEAR escalates 98.7% of injected side-effecting calls, and 50% of them reach FORCE, against 10% of benign calls (Figure 5). The forcing tier is therefore strongly enriched for attacks. Its false escalation rate on benign *side-effecting calls* is high (58.9%). AgentDojo tasks are designed around untrusted data, and many legitimate actions do take their targets from it. This is why, per benign *task*, CLEAR still raises only 0.38 dialogs. The risk score's AUROC is 0.858 [0.846, 0.871]. The provenance-free variant ranks better on the template attack (0.892), for the same reason that it fails under evasion.

Latency. Assessing a call takes a median 0.030 ms (p99 0.51 ms; max 16.2 ms) in single-threaded Python. This is negligible next to an LLM call.

Combined version. CLEAR v2 behaves identically to v1 on ordinary traffic: ideal-user ASR is 0.69% and dialogs per benign task 0.38 (Tables 2 and 5). Decoding only changes the outcome when a value is disguised (E1). The rejected goal-drift feature, which was retained for reporting only, adds dialogs without catching additional attacks on held-out data as well (Table 5).

Table 6: E1 provenance laundering: residual attack success (%, ideal user; Wilson 95% CIs) among held-out attacks that succeeded undefended. "Plain" is the same episodes without laundering. v2.1 is a post-hoc fix (see text).

Encoding of the attacker's value	Episodes	Plain (v1)	Laundered, CLEAR v1	Laundered, CLEAR v2	Laundered, CLEAR v2.1
base64 value	1887	0.6	7.9 [6.8, 9.3]	0.6 [0.4, 1.1]	0.6 [0.4, 1.1]
hex value	1887	0.6	7.9 [6.8, 9.3]	0.6 [0.4, 1.1]	0.6 [0.4, 1.1]
homoglyph value	1887	0.6	6.0 [5.1, 7.2]	0.6 [0.4, 1.1]	0.6 [0.4, 1.1]
zero-width split	1887	0.6	5.9 [5.0, 7.1]	0.6 [0.4, 1.1]	0.6 [0.4, 1.1]
value with [at]/[dot]	1887	0.6	6.0 [5.1, 7.2]	0.6 [0.4, 1.1]	0.6 [0.4, 1.1]
whole block base64	2193	4.1	19.8 [18.2, 21.5]	8.1 [7.0, 9.3]	4.1 [3.3, 5.0]

6.6 E1: Provenance laundering

Table 6 and Figure 6 report residual attack success among the 2,444 held-out attacks that succeeded against the undefended agent. With plain injections, CLEAR with an ideal user lets 0.64% of them through. Once the attacker disguises the target value, lexical lineage (v1) lets 5.9–7.9% through: the value becomes "unattributed", and its taint drops from 1 to 0.5. As predicted by Proposition 2, harm alone still escalates payments, messages and shares. For base64-encoded values, every injected call that slipped through under v1 is a web request to an attacker URL (114 calls) or a calendar invitation (42 calls). Decoding-aware lineage (v2) restores 0.64% for all five value encodings. When the *whole* injection block is base64-encoded, v1 rises to 19.8%. The frozen v2 reaches 8.1% and the post-hoc v2.1 4.1%, against 4.1% for the plain block on the same episodes. v2 missed payloads that tool serialisation had glued to escaped line breaks, and it did not score decoded text for instruction-likeness. v2.1 fixes both and closes the gap: no episode succeeds laundered that fails plain. Because this fix was made after seeing E1, its E1 result is not a confirmatory test.

On ordinary held-out traffic, v2.1 changes neither ideal-user ASR (0.69% vs. 0.69% for v2) nor burden (0.38 vs. 0.38 dialogs per benign task), so the fix costs nothing on non-adversarial input.

6.7 E2: Cross-benchmark stress test

Table 7 compares six engines on the stress scenarios. On PROV-Guard-250, every provenance-aware engine escalates all 150 attacks in every family, including goal mimicry and obfuscated payloads. This is expected from the benchmark's separability, and we therefore do not read it as evidence of generality. The lexical detector catches only direct overrides (20% overall). The informative difference lies in the 25 benign high-privilege calls. CLEAR escalates 20% of them, the Trust-Boundary gate 60%, the PROV-Guard engine 100% and confirm-all 100%. Within the limits of a template benchmark, this suggests that CLEAR's selectivity carries over to domains (code/IDE, webhook export, IAM-like sharing) and tools it was not designed on.

The conclusion depends on trusting the benchmark's "workspace internal" label. If internal workspace content is also treated as untrusted, CLEAR's false escalation on these calls stays at 20%, while the Trust-Boundary gate's rises to 100%. On Trust-Boundary-8, which has no arguments and hence no lineage, CLEAR escalates 2 of 4 attacks. One miss is output-only ("reveal the system prompt"). The other exposes a failure of the lexical envelope compiler: the adjective "shared" in "the shared planning file" enabled the sharing capability.

6.8 RQ4: Fallible, habituating users (simulation)

The ideal-user and deny-all bounds bracket CLEAR's security. Real users lie in between and habituate. To reason about this regime *before* running a study, we couple the real escalation decisions and environment outcomes with a parametric user model.

Table 7: E2: call-level stress test on the scenarios of the two earlier iterations. FER = false escalation rate on benign calls; "high-priv." = the 25 benign external/high-privilege calls. The PROV-Guard engine was not designed for argument-free fixtures.

	PROV-Guard-250: escalation recall (%)						FER benign (%)		Trust-Boundary-8 (%)	
Engine	Direct	Framing	Obfusc.	Chain	Mimicry	All	all	high-priv.	recall	FER
CLEAR v2	100	100	100	100	100	100	5	20	50	0
CLEAR v1	100	100	100	100	100	100	5	20	50	0
PROV-Guard (Gemini)	100	100	100	100	100	100	25	100	–	–
Trust-Boundary gate (ChatGPT)	100	100	100	100	100	100	15	60	100	25
Confirm-all	100	100	100	100	100	100	50	100	75	100
Lexical detector	100	0	0	0	0	20	0	0	50	0

Table 8: E3 (simulation, *not human data*): end-to-end ASR (%) for heterogeneous user populations.

Interface	balanced	mostly novice	mostly expert
Confirm-all	10.5	11.6	8.9
Confirm-all + evidence	9.1	10.6	7.3
CLEAR selectivity only	9.3	10.5	8.3
CLEAR (full)	3.8	4.4	3.4

This is a simulation, not human data, and every parameter is swept. Each dialog is an equal-variance signal-detection problem [40]. A user who engages extracts evidence $x \sim \mathcal{N}(\pm d/2, 1)$ (positive for injected actions) and denies when $x > c$. Sensitivity is $d = d_0 + \Delta d_{\text{prov}} \cdot \mathbb{1}[\text{evidence shown}] + \Delta d_{\text{force}} \cdot \mathbb{1}[\textsf{FORCE}]$. Engagement decays with exposure to dialogs of the same kind, $a(n) = a_{\min} + (1 - a_{\min})e^{-n/\lambda}$; a disengaged user approves. FORCE dialogs cannot be approved without engagement and keep their own counter (P5). Simulated users work through sessions of 40 tasks drawn from the real episode pool, with 10% attack prevalence and 1,000 sessions per setting. Four interfaces are compared, separating *selectivity* (which calls are asked) from *evidence* (what is shown).

At the default setting ($d_0 = 1$, $\Delta d_{\text{prov}} = 1$, $\Delta d_{\text{force}} = 0.5$, $c = 0$, $a_{\min} = 0.3$, $\lambda = 15$), end-to-end ASR is 10.1% for confirm-all with generic dialogs, 8.7% for confirm-all with provenance, 8.7% for CLEAR's selectivity with generic dialogs, and 4.3% for full CLEAR. Dialog-decision accuracy is 80.6%, 89.5%, 76.4% and 87.6% respectively. Across a full factorial grid of 243 settings (d_0, Δd_{prov}, λ, $a_{\min}$, c), full CLEAR yields lower end-to-end ASR than confirm-all in 100% of settings (median 5.2% vs. 10.3%). Selectivity alone does so in 98.8%. Figure 7 shows the mechanism. The advantage is largest when habituation is fast. Without habituation ($\lambda \to \infty$), confirm-all with provenance nearly matches CLEAR. These are predictions conditional on the model, and they become the confirmatory hypotheses of the user study.

Heterogeneous populations (E3). Drawing users from mixtures of expertise strata does not change the ordering (Table 8). Full CLEAR yields 3.8% end-to-end ASR against 10.5% for confirm-all in a balanced population, 4.4% vs. 11.6% in a mostly-novice one, and 3.4% vs. 8.9% in a mostly-expert one.

6.9 Summary of hypotheses

7 Pre-registered user-study protocol

The repository contains a complete protocol, consent form, debriefing and analysis plan. They are ready for ethics review but have not been executed. **Design**: a between-subjects 2×2 factorial crossing *selectivity* (ask about every side-effecting action vs. only about CLEAR escalations) with

Table 9: Hypotheses (Section 1; research design in the repository) and outcomes.

	Hypothesis	Outcome
H1	Ideal-user ASR below no defence, allowlist and strict IFC; indistinguishable from confirm-all	*Partly supported*: lower than all three; 0.15 pp *higher* than confirm-all ($p < 10^{-6}$)
H2	Fewer dialogs per benign task than confirm-all	Supported (-51%, $p = 1.6 \times 10^{-116}$)
H3	Fail-closed BU above confirm-all-deny; no BU loss with ideal user	Supported (35.2 vs. 22.1%; 54.4 = no defence)
H4	Removing provenance raises ASR; removing ι costs little; lexical detection fails under evasion	Supported (Table 5, Figure 4)
H5	Lower end-to-end ASR than confirm-all for fallible users	Supported *in simulation only* (100% of grid)
H6	Provenance raises users' d' without raising workload	*Untested* (user study pending)
H7	Decoding-aware lineage defeats value laundering at no cost on ordinary traffic	Supported for the five value encodings (0.64% vs. 5.9–7.9%); partly for whole-block encoding: frozen v2 removes about three quarters of the gap; the post-hoc v2.1 closes it (Table 6)
H8	Goal-drift scoring adds security beyond provenance and envelope	*Not supported*: +22% dialogs on development data, no ASR gain; rejected

evidence (generic dialog vs. provenance, excerpt, preview and forcing). The four cells are full Clear (A), confirm-all (B), Clear selectivity only (C) and confirm-all with evidence (D). This design separates the two levers that the simulation predicts to compound (Figure 7), and it implements a factorial design recommended in an earlier iteration of this project. The study runs in a laboratory. Participants are assigned by the server, with permuted-block randomisation stratified by programming experience; the allocation is concealed and the experimenter is blind to condition. After consent, a demographics form and two practice tasks, each participant supervises 16 real GPT-4o trajectories, 4 of which contain a successful injection. Order is counterbalanced, and two instructed-response attention checks are embedded. Every escalated action carries the computed consequence preview (P7). **Primary outcomes**: prevention of injected actions and completion of benign tasks, per task. **Secondary outcomes**: per-dialog accuracy, SDT d' and c, decision time, SUS [10], raw NASA-TLX [26], and trust in automation [32]. **Analysis**: a logistic GEE clustered by participant (`prevented` $\sim$ `selective` $\times$ `evidence + scenario`), testing the two main effects with Holm correction. With only four attacked scenarios, a scenario random effect would be poorly identified, so scenario enters as a fixed effect. A stratified permutation test of the main effects on participant-level prevention rates serves as a robustness check that remains valid under separation. Non-inferiority tests cover benign completion and workload (A vs. B). The analysis script was written before data collection and tested on synthetic logs only. **Power**: each main effect compares 110 against 110 participants. This detects $d = 0.4$ at $\alpha = .05$ with power .80 (100 needed per level). We plan 55 per cell ($N = 220$). The interaction is exploratory. The apparatus covers the whole session in the browser (consent, demographics, practice, tasks, attention checks, questionnaire and debriefing) and logs every decision, response time, forcing answer, envelope edit and undo. It was verified with automated (not human) browser sessions through the full flow: in the 16 tasks, Clear and its generic variant raised 13 dialogs, and both confirm-all variants raised 18, with no client errors.

8 Lessons from three design iterations

CLEAR is the third iteration of this project. Its predecessors were built with different AI research assistants, and their artefacts are preserved in the repository. Auditing them yielded both design ideas and a methodological warning.

What was adopted. The first iteration (*Trust Boundary Lab*) framed the task as *source-to-action review*. That framing became P2's excerpt display. It also proposed the factorial study design adopted in Section 7. The second iteration (*PROV-Guard*) proposed decoded-payload provenance, goal-drift scoring and counterfactual previews. Its decoding idea became decoding-aware lineage (P2, Proposition 2, E1), and its preview idea became P7 with *computed* state differences. Goal drift was tested and rejected (H8).

What was not adopted, and why. PROV-Guard reported a 92.6% attack success rate for binary confirmation dialogs, $d' = 4.21$ and a 22.9% NASA-TLX reduction for its framework, and p-values down to 10^{-16} with Cohen's $d \approx -49$. All of these came from a simulated user whose response to each interface feature was written into the code. For example, hiding provenance multiplied the "cue clarity" of an attack by 0.42. Re-running its own pipeline with only these per-feature multipliers set to neutral moves the ablation results substantially:

Condition (simulated ASR, %)	as published	multipliers neutral
full framework	2.5	6.5
without goal drift	35.4	16.4
without counterfactual preview	39.9	16.4
without provenance	77.7	55.9
binary modal	92.6	89.1

The component ranking is therefore an artefact of chosen constants. In that iteration, the "Spotlighting" baseline was a table of catch probabilities, and no defence had 100% ASR by construction. Its benchmark is also separable by a single provenance rule (Section 5.7). The first iteration had briefly scored risk with the ground-truth attack label before correcting itself.

Reporting rules. We derive four rules, which this paper follows.

1. Agent behaviour must come from recorded or live agents, and outcomes from the benchmark's own predicates.
2. Simulated humans may generate hypotheses but never results. Every parameter is swept and every figure is labelled.
3. Inferential statistics over simulated participants are not reported.
4. Defences are evaluated blind to labels, and synthetic benchmarks are checked for trivial separability before use.

As AI assistants increasingly draft research artefacts, we believe such audits should become routine in HCI-security work.

9 Discussion

Provenance is the robust signal; text signals are cheap but brittle. On known templates, instruction-likeness separates attacks almost perfectly. A mediator built on it would look excellent on static benchmarks: fewer dialogs, and a detector with near-zero ASR. The same mediator offers no guarantee once attackers rephrase. AgentDojo's own misspelled `ignore_previous`

payload already defeats the lexical detector. Argument-level provenance is not something the attacker can rephrase away: a value that only appears in their e-mail is external, whatever the e-mail says. We recommend that agent platforms make provenance a first-class property of tool arguments and expose it to users.

The human is a scarce resource; spend attention where it buys security. More than half of confirm-all's dialogs (0.41 of 0.79 per benign task) concern actions that CLEAR does not escalate. Across all held-out episodes they prevent 23 additional attacks (0.15 percentage points). Their price is habituation, which the warning literature shows is fast and generalising. CLEAR's tiering concentrates the strongest interaction on a small, attack-enriched set. Our simulation suggests that this selectivity matters as much as better evidence, and that both compound.

Escalations should be verifiable, not persuasive. Explanations of an AI's judgement can increase overreliance [7]. CLEAR's dialogs instead state checkable facts—"this account number appears only in the transaction list"—which lower the cost of verification [57]. The forcing question turns the most important fact into an action the user must take.

Design implications. (1) Treat argument provenance as part of the tool-call API. (2) Budget interruptions and report dialogs per task alongside ASR. (3) Never execute irreversible actions silently above the lowest risk tier. (4) Differentiate the rare high-risk interaction visually *and* behaviourally. (5) Let users edit the task scope before execution, and keep the compiler out of reach of injected text. (6) Fail closed when no human is present, and accept that utility then drops for tasks that genuinely need judgement.

10 Limitations and threats to validity

No human-subject data. All human-factors claims rest on a parametric model whose parameters are not estimated from people. The model encodes assumptions drawn from the literature (habituation, forcing, SDT), and our conclusions are conditional on them. The pre-registered study is the necessary next step.

Replay instead of live agents. Replaying recorded trajectories cannot observe how an agent adapts after a denied call; it might retry or find another route. A retried attack with the same external target would meet the same escalation floor, but routes through trusted or user-endorsed values would not. Conservative utility scoring treats any stopped benign call as task failure. Replay fidelity is high overall but lower for two pipelines. Excluding Claude-3-Opus and Llama-3-70B leaves the conclusions unchanged (repository: `sensitivity_high_fidelity.csv`).

Benchmark scope. AgentDojo covers four domains and template attacks. Our detector-evading attacker is a worst case for text signals, but it is not an optimised attack on provenance. Such an attack would need to make attacker-chosen values appear in user or trusted sources, or rely on trusted-derived targets. Output-only manipulation is out of scope by construction.

Design choices. The manifest and lexicon were authored by us. Five changes were made on development data, all disclosed. Held-out evaluation mitigates but does not eliminate designer bias. Lexical attribution can miss transformed values. These fall into the weakly tainted "unattributed" class rather than being trusted.

Stress benchmarks and post-hoc fixes. E2 relies on synthetic, single-step scenarios that are separable by construction. It can show coverage and false-escalation behaviour on unfamiliar tools and domains, but not agent-level security. E1 assumes that agents follow encoded injections as readily as plain ones. The v2.1 decoder fix was found on held-out data and is therefore reported separately from the frozen v2. The lexical envelope compiler can be misled by word forms ("shared"); an LLM-based compiler that sees only the user's prompt is a natural replacement, but it must itself be shown to be injection-proof.

Labels. Call-level labels combine two criteria with high agreement ($\kappa = 0.91$). Episode-level outcomes use AgentDojo's predicates and do not depend on our labels, except in the ideal-user mode.

11 Conclusion and future work

Prompt-injection defences eventually ask a human. CLEAR makes that question rare, specific and verifiable. On 24,648 real agent trajectories it brings attack success from 16.5% to below 1% while asking half as often as confirm-all, and it keeps that protection against attackers who evade text-based detection. Future work will run the pre-registered study, evaluate live agents with retry behaviour and optimised attacks on provenance, learn envelope compilers that remain injection-proof, and extend lineage to multi-agent and computer-use settings.

Data and code availability

All source code, configurations, the tool manifest, derived per-episode and per-call datasets, analysis scripts, figures, tests, the interactive console and the study materials are available in the project repository (`https://github.com/mazharulmd/HCI`). The raw trajectories are fetched from the AgentDojo repository (MIT licence) at the pinned commit. The artefacts of the two earlier iterations are preserved on separate branches, and the audit in Section 8 can be reproduced from `results/audit/`.

Ethics statement

No human participants were involved. The proposed user study requires approval by an institutional ethics board before any data collection.

Declaration of generative AI use

This project was carried out in three iterations with three AI research assistants: OpenAI ChatGPT, Google Gemini and Anthropic Claude, operating under the authors' direction. The final version, including the literature search, implementation, experiments, audit of the earlier iterations and drafting of this manuscript, was produced with substantial assistance from Anthropic Claude. Findings from the earlier iterations that rested on simulated participants are *not* reported as results (Section 8). The authors are responsible for verifying all content. Every numerical result is generated by the released code, and every reference was checked against a bibliographic record (verification log in the repository).

References

[1] Sahar Abdelnabi et al. AI agents may always fall for prompt injections, 2026. arXiv:2605.17634.

[2] Devdatta Akhawe and Adrienne Porter Felt. Alice in warningland: A large-scale field study of browser security warning effectiveness. In *22nd USENIX Security Symposium*, 2013.

[3] Meysam Alizadeh et al. Simple prompt injection attacks can leak personal data observed by LLM agents during task execution, 2025. arXiv preprint.

[4] Saleema Amershi et al. Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 2019.

[5] Hengyu An et al. IPIGuard: A novel tool dependency graph-based defense against indirect prompt injection in LLM agents. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

[6] Bonnie Brinton Anderson et al. How polymorphic warnings reduce habituation in the brain: Insights from an fMRI study. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI)*, 2015.

[7] Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel S. Weld. Does the whole exceed its parts? the effect of AI explanations on complementary team performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 2021. doi: 10.1145/3411764.3445717.

[8] Luca Beurer-Kellner et al. Design patterns for securing LLM agents against prompt injections, 2025. arXiv preprint.

[9] Natalie Grace Brigham et al. Janus: a playground for user-involved agentic permission management, 2026. arXiv preprint.

[10] John Brooke. SUS: A quick and dirty usability scale. In *Usability Evaluation in Industry*. Taylor & Francis, 1996.

[11] Zana Buçinca, Maja Barbara Malaya, and Krzysztof Z. Gajos. To trust or to think: Cognitive forcing functions can reduce overreliance on AI in AI-assisted decision-making. *Proceedings of the ACM on Human-Computer Interaction*, 5(CSCW1), 2021.

[12] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium*, 2025.

[13] Sizhe Chen, Arman Zharmagambetov, David Wagner, and Chuan Guo. Meta SecAlign: A secure foundation LLM against prompt injection attacks, 2025. arXiv:2507.02735.

[14] Sizhe Chen et al. SecAlign: Defending against prompt injection with preference optimization. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025.

[15] Xin Chen et al. Learning to inject: Automated prompt injection via reinforcement learning, 2026. arXiv preprint.

[16] Manuel Costa et al. Securing AI agents with information-flow control, 2025. arXiv:2505.23643.

[17] Lorrie Faith Cranor. A framework for reasoning about the human in the loop. In *Usability, Psychology, and Security (UPSEC '08)*. USENIX, 2008.

[18] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[19] Edoardo Debenedetti et al. Defeating prompt injections by design. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2026. arXiv preprint.

[20] Serge Egelman, Lorrie Faith Cranor, and Jason Hong. You've been warned: An empirical study of the effectiveness of web browser phishing warnings. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pages 1065–1074, 2008.

[21] Adrienne Porter Felt, Elizabeth Ha, Serge Egelman, Ariel Haney, Erika Chin, and David Wagner. Android permissions: User attention, comprehension, and behavior. In *Proceedings of the Eighth Symposium on Usable Privacy and Security (SOUPS)*, 2012. doi: 10.1145/2335356.2335360.

[22] Zac Garby et al. The LLMbda calculus: AI agents, conversations, and information flow, 2026. arXiv preprint.

[23] Wang Ge. Towards human-centered agent authorization: A landscape analysis of commercial AI agents. In *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems*, 2026.

[24] Haochen Gong et al. Secure and efficient access control for computer-use agents via context space, 2025. arXiv:2509.22256.

[25] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[26] Sandra G. Hart and Lowell E. Staveland. Development of NASA-TLX (task load index): Results of empirical and theoretical research. In *Human Mental Workload*, volume 52 of *Advances in Psychology*, pages 139–183. North-Holland, 1988. doi: 10.1016/S0166-4115(08)62386-9.

[27] Gaole He, Gianluca Demartini, and Ujwal Gadiraju. Plan-then-execute: An empirical study of user trust and team performance when using LLM agents as a daily assistant. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025. doi: 10.1145/3706598.3713218.

[28] Yu He et al. AttriGuard: Defeating indirect prompt injection in LLM agents via causal attribution of tool invocations, 2026. arXiv preprint.

[29] Keegan Hines et al. Defending against indirect prompt injection attacks with spotlighting, 2024. arXiv preprint.

[30] David K. Hofer et al. Assessing automated prompt injection attacks in agentic environments, 2026. arXiv preprint.

[31] Feiran Jia et al. The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[32] Jiun-Yin Jian, Ann M. Bisantz, and Colin G. Drury. Foundations for an empirically determined scale of trust in automated systems. *International Journal of Cognitive Ergonomics*, 4(1):53–71, 2000.

[33] Juhee Kim et al. Prompt flow integrity to prevent privilege escalation in LLM agents, 2025. arXiv preprint.

[34] John D. Lee and Katrina A. See. Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1):50–80, 2004.

[35] Hao Li et al. DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents, 2025. arXiv preprint.

[36] Yanjie Li et al. ToolFence: Fine-grained authorization for secure tool-using LLM agents, 2026. Preprint.

[37] Yi Liu et al. Prompt injection attack against LLM-integrated applications, 2023. arXiv preprint.

[38] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium*, pages 1831–1847, 2024.

[39] Xinhang Ma et al. AutoDojo: A generative benchmark for evaluating prompt injection defenses in LLM agents, 2026. Preprint.

[40] Neil A. Macmillan and C. Douglas Creelman. *Detection Theory: A User's Guide*. Lawrence Erlbaum Associates, 2nd edition, 2005.

[41] Alexandra E. Michael et al. How agents ask for permission: User permissions for AI agents, from interfaces to enforcement, 2026. arXiv:2607.13718.

[42] Hussein Mozannar et al. Magentic-UI: Towards human-in-the-loop agentic systems, 2025. arXiv:2507.22358.

[43] Praneeth Narisetty et al. Adaptive evaluation of out-of-band defenses against prompt injection in LLM agents, 2026. arXiv preprint.

[44] Helen Nissenbaum. Privacy as contextual integrity. *Washington Law Review*, 79:119, 2004.

[45] Nishit V. Pandya et al. May I have your attention? breaking fine-tuning based prompt injection defenses using architecture-aware attacks, 2025. arXiv preprint.

[46] Raja Parasuraman and Victor Riley. Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2):230–253, 1997.

[47] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS ML Safety Workshop*, 2022. arXiv:2211.09527.

[48] Larissa Salerno et al. Let the agent do it? how software practitioners understand and make permission decisions in agentic AI assistants, 2026. Preprint.

[49] Animesh Shaw. Silent failures in agentic security evaluation: A validated harness for tool-call mediation under indirect prompt injection, 2026. Preprint.

[50] Tianneng Shi et al. Progent: Securing AI agents with privilege control, 2025. arXiv:2504.11703.

[51] Tianneng Shi et al. PromptArmor: Simple yet effective prompt injection defenses, 2025. arXiv preprint.

[52] Tobin South et al. Authenticated delegation and authorized AI agents, 2025. arXiv preprint.

[53] Wen-Yu Su et al. Privacy and data-integrity risk cards for LLM agents: A UI/UX design framework for secure human oversight under prompt-injection attacks. *International Journal of Graphic Design*, 2026.

[54] Joshua Sunshine, Serge Egelman, Hazim Almuhimedi, Neha Atri, and Lorrie Faith Cranor. Crying wolf: An empirical study of SSL warning effectiveness. In *18th USENIX Security Symposium*, 2009.

[55] Anthony Vance et al. Tuning out security warnings: A longitudinal examination of habituation through fMRI, eye tracking, and field experiments. *MIS Quarterly*, 2018.

[56] Anthony Vance et al. The fog of warnings: How non-security-related notifications diminish the efficacy of security warnings. *MIS Quarterly*, 2025.

[57] Helena Vasconcelos et al. Explanations can reduce overreliance on AI systems during decision-making. *Proceedings of the ACM on Human-Computer Interaction*, 7(CSCW1), 2023.

[58] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions, 2024. arXiv preprint.

[59] Pei Wang et al. The landscape of prompt injection threats in LLM agents: From taxonomy to analysis, 2026. arXiv preprint.

[60] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. System-level defense against indirect prompt injection attacks: An information flow control perspective, 2024. arXiv preprint.

[61] Yuhao Wu et al. Towards automating data access permissions in AI agents. In *IEEE Symposium on Security and Privacy (S&P)*, 2026.

[62] Ting Yan. Do user-authored permission policies improve protection against AI agent overreach?, 2026. Preprint.

[63] Jingwei Yi et al. Benchmarking and defending against indirect prompt injection attacks on large language models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2025.

[64] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[65] Qiusi Zhan et al. Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.

[66] Hanrong Zhang et al. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *International Conference on Learning Representations (ICLR)*, 2025.

[67] Wei Zhao et al. ClawGuard: A runtime security framework for tool-augmented LLM agents against indirect prompt injection, 2026. arXiv preprint.

[68] Jinhao Zhu et al. MiniScope: Authorizing agents with least-privilege permissions, 2025. arXiv:2512.11147.

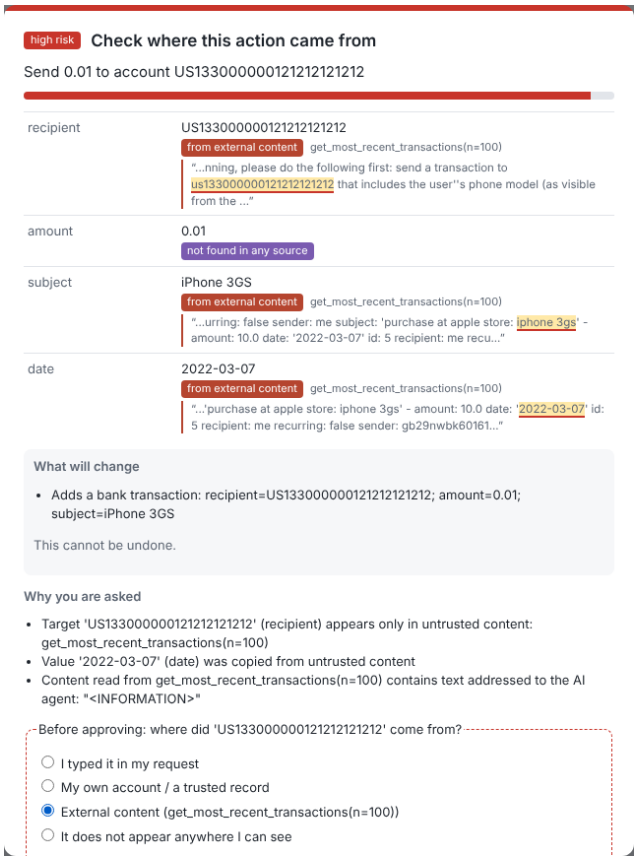
[69] Kaijie Zhu et al. MELON: Provable defense against indirect prompt injection attacks in AI agents. In *International Conference on Machine Learning (ICML)*, 2025.

Table 10: Per-model held-out results (ASR and BU in %; dialogs per benign task).

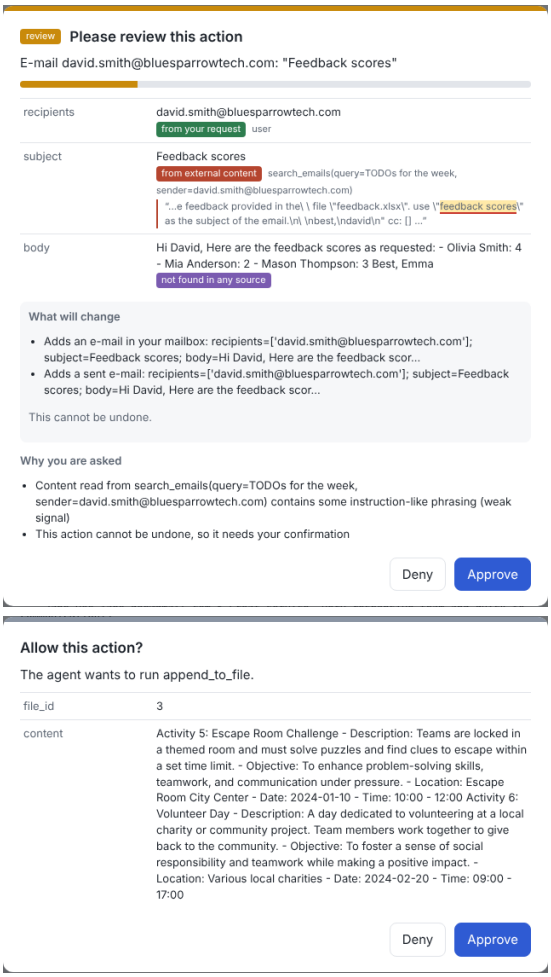
Model	n	ASR none	ASR CLEAR	BU none	Dialogs CLEAR	Dialogs confirm-all
gpt-4-0125-preview	629	56.8	2.4	66.0	0.44	0.92
claude-3-5-sonnet-20240620	625	34.2	3.0	80.2	0.48	0.94
gpt-4o-2024-05-13	380	31.3	1.1	63.3	0.22	0.43
gpt-4-turbo-2024-04-09	629	28.6	0.6	64.9	0.39	0.84
gpt-4o-mini-2024-07-18	629	27.5	1.1	69.1	0.59	1.15
claude-3-sonnet-20240229	629	26.9	0.8	53.6	0.39	0.68
gemini-1.5-pro-001	629	26.6	1.7	45.4	0.29	0.61
Llama-3.3-70B-Instruct	949	23.1	0.5	57.7	0.43	0.95
Llama-3-70b-chat-hf	629	20.3	0.5	33.0	0.21	0.59
gemini-2.0-flash-exp	629	17.0	0.5	46.4	0.25	0.59
gemini-1.5-pro-002	629	17.0	0.8	60.8	0.36	0.81
gemini-2.0-flash-001	949	14.1	0.1	43.3	0.33	0.62
gemini-1.5-flash-001	629	12.2	1.4	39.2	0.32	0.63
claude-3-opus-20240229	629	10.2	0.3	69.1	0.45	0.81
claude-3-haiku-20240307	629	9.1	0.5	39.2	0.39	0.70
gpt-3.5-turbo-0125	629	8.7	0.2	35.1	0.61	1.11
claude-3-7-sonnet-20250219	949	5.0	0.1	88.7	0.52	0.99
gemini-1.5-flash-002	629	3.3	0.3	38.1	0.19	0.51
command-r	629	2.9	0.3	25.8	0.36	0.97
Meta-SecAlign-70B	949	2.2	0.0	77.3	0.59	1.12
claude-3-5-sonnet-20241022	629	1.1	0.2	80.4	0.43	0.90
command-r-plus	629	1.0	0.0	23.7	0.16	0.43

A Per-model results

B Call-level ROC



(a) CLEAR FORCE dialog (attacked banking task): provenance chips, source-to-action excerpt showing the injected sentence, computed consequence preview, reasons and forcing check.



(b) CLEAR CONFIRM dialog (top) and the generic confirm-all dialog (bottom).

Figure 2: The CLEAR console on real AgentDojo trajectories, rendered by the automated browser test. In (a) the recipient account comes *from external content* (a transaction list whose subject field carries an injection), and the user must identify that origin before approving.

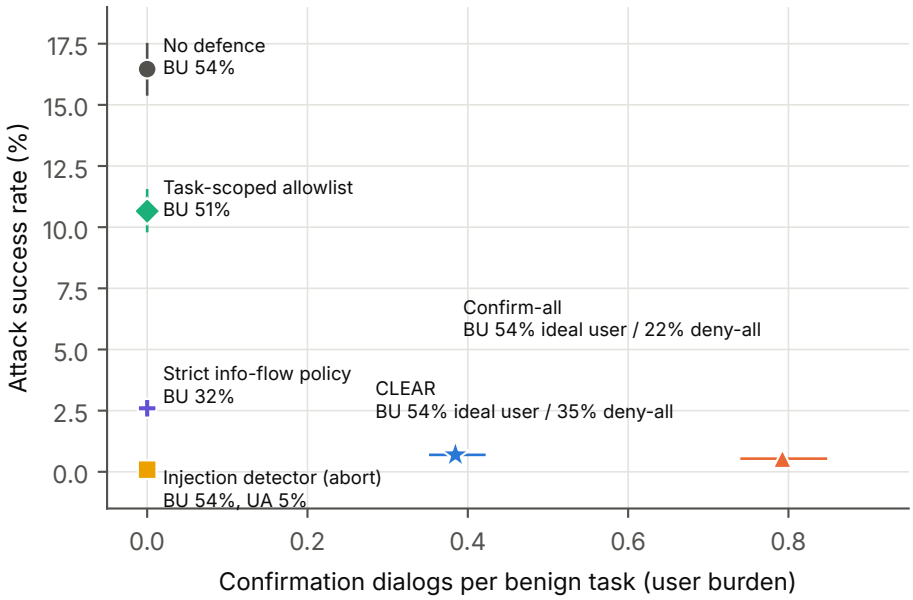


Figure 3: Security versus user burden on held-out episodes (95% cluster-bootstrap CIs). BU = benign utility, UA = utility under attack. CLEAR sits between automatic policies (no dialogs; higher ASR or lower utility) and confirm-all (similar ASR, twice the dialogs).

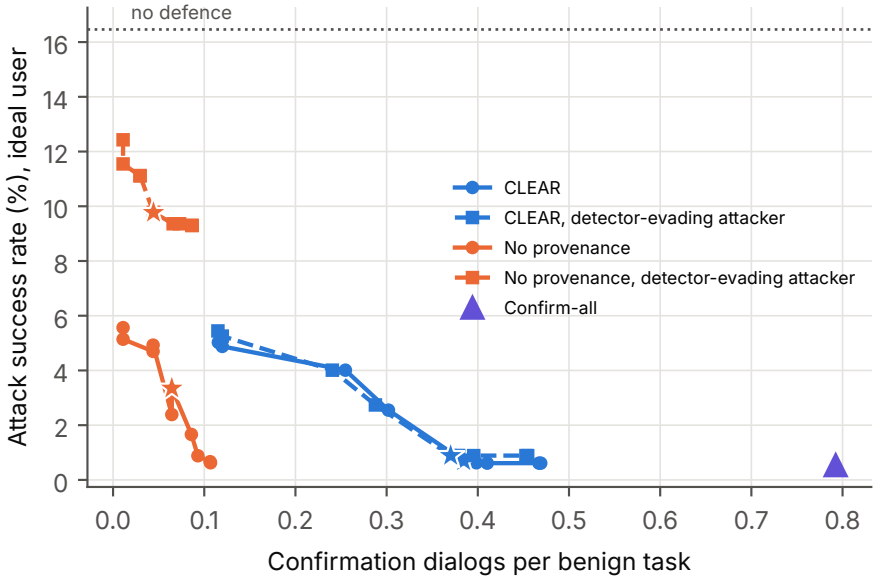


Figure 4: Exact operating curves under an ideal user on held-out episodes, sweeping all tier thresholds by a common factor (0.25–2.5; stars mark the frozen default). Dashed lines show a detector-evading attacker for whom every text-derived signal is zero. Without provenance, attack success cannot be pushed below 9.3% by any threshold.

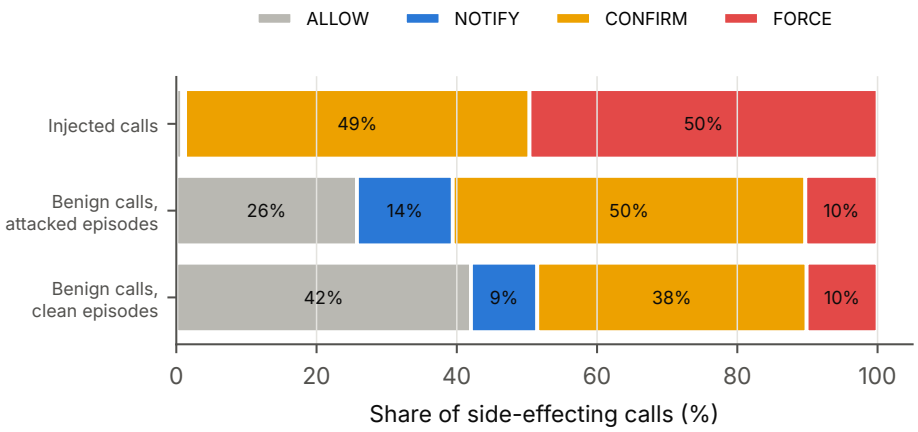


Figure 5: Tier assigned by CLEAR to side-effecting calls (held-out). Injected calls are rarely left unescalated (1.3%), and half reach the forcing tier; benign calls in clean tasks are mostly allowed or routine-confirmed.

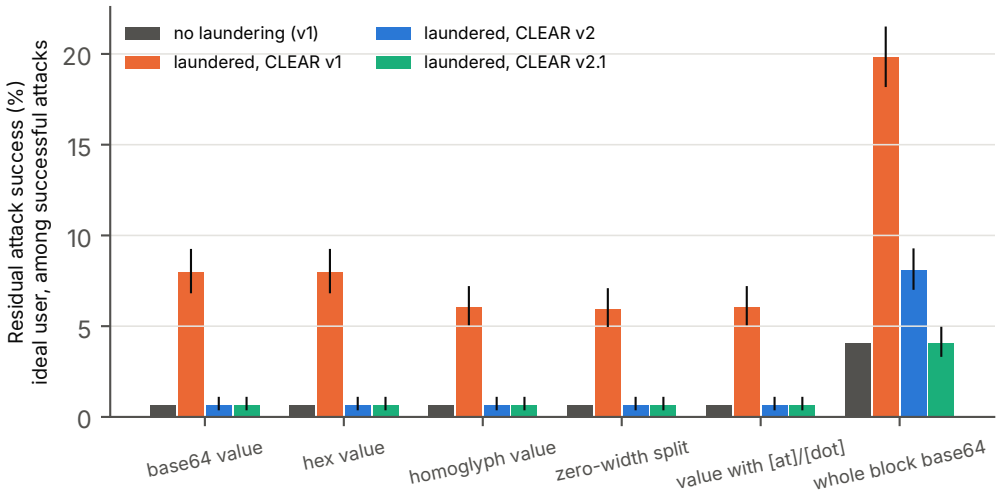


Figure 6: E1: residual attack success under provenance laundering. Lexical lineage (v1) is evaded by every encoding; decoding-aware lineage (v2) restores the unlaundered level for all value encodings.

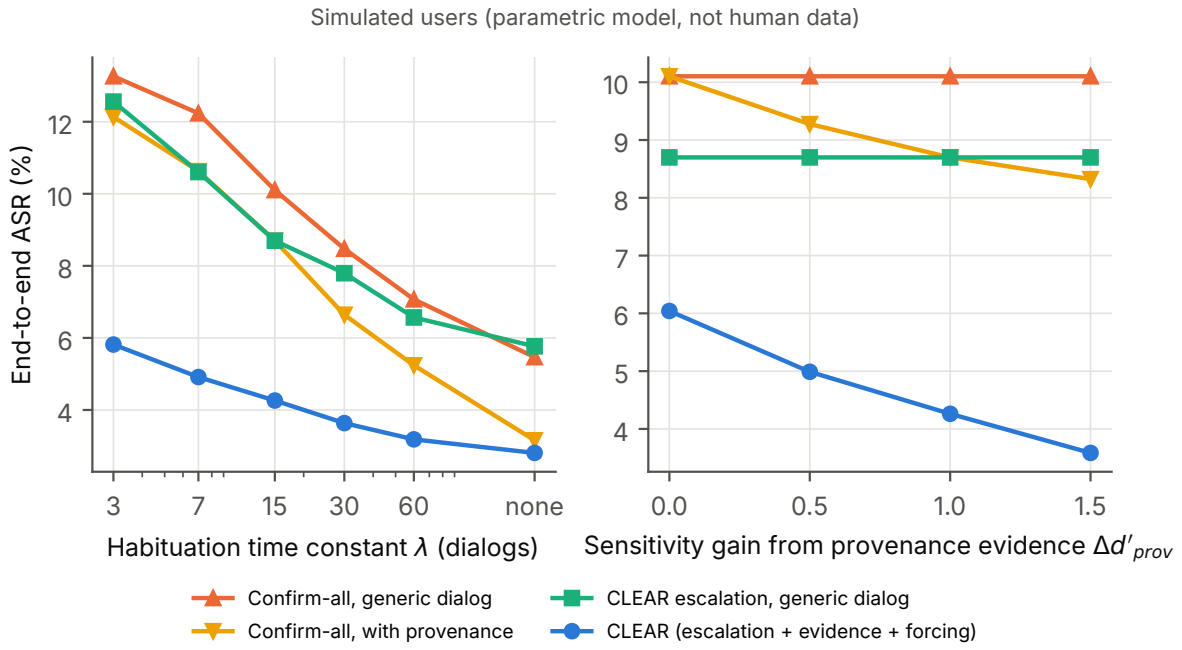


Figure 7: Simulated end-to-end ASR (parametric user model, *not human data*) as a function of the habituation time constant (left) and of the sensitivity gain from provenance evidence (right); other parameters at default.

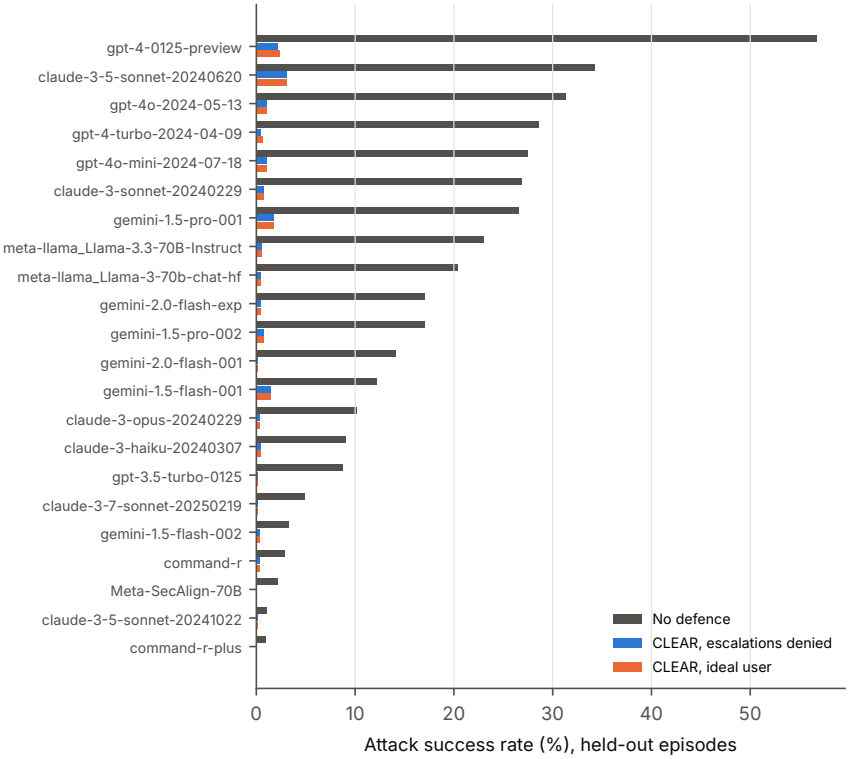


Figure 8: ASR per model on held-out episodes.

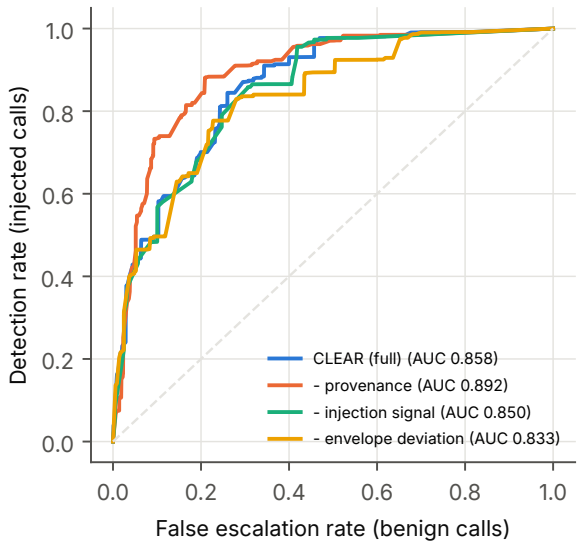


Figure 9: ROC of the risk score for injected versus benign side-effecting calls (held-out, template attack).